

方案：AI 微信群 · 需求分析 / 产品 / 技术方案

现状是一个一次性圆桌引擎（见 [技术方案-代码编排与核心逻辑.md](#)）。

本文记录把它扩展成「AI 微信群」的方案讨论——需求、产品、技术、已敲定的决策与待定项。

状态：方案讨论阶段，尚未写代码。配套：[后端排查记录-opencode-go.md](#)。

0. 愿景

建一个「AI 微信群」：

- 每个 AI 是一个**有持久身份的好友**；
- 把若干好友拉进一个**群**；
- 你提出问题 → **主持人划分讨论维度** → 群里 AI 每人分一个维度做圆桌讨论；
- 你也是群里的**平等参与者**，能随时插话、追问、改方向。

最小实现 = 对话式多 agent 编排。

远期方向 = 按人类管理架构（经理 → 下属、群套群）在聊天软件里管理多 agent。

1. 核心转变：从"流水线"到"群聊"

现有 RoundTable 是一条**一次性流水线**：人丢进事件 → 策划/陈述/质询/主笔跑完 → 出结论，人全程旁观。

「AI 微信群」是一个**开放的、人在环里的对话空间**：人随时插话，AI 之间持续你来我往。

这不是 UI 换皮，是编排内核的变化——从**"固定阶段机"变成"谁该下一个说话的调度循环"**。这是整个方案最硬的骨头。

现有四个角色（策划/主持人/视角/主笔）和那套 prompt **不丢弃，而是降级成"主持人可随时调用的招式"**：要不要先 framing+分维度、讨论乱了要不要收、该不该叫主笔出结论。原来写死的 plan→open→debate→synthesis，变成主持人手里的工具箱。

2. 需求分析

2.1 实体（名词）

实体	说明	现状
好友 / AI Contact	有 持久身份 的 agent（头衔、人设、记忆）	✗ 现在视角临时生成，无身份无记忆
群 / Group	持久的多 agent + 人的对话空间，有成员、历史、主题	✗ 无"群"概念，跑完即弃
主持人	划分维度、分配发言、控节奏	☑ 有，但只在固定流程里
消息 / Message	群里每条发言（人或 AI），带时间线	⚠ 有 Turn，但只读，人不能发
你（人类）	群里的 平等参与者	✗ 现在只能开场喂一次事件

2.2 新增的硬需求（动词）

1. **人可中途插话**：讨论进行中，人发消息，AI 要接住并回应。
2. **发言权调度**：开放群聊里"下一个谁说话"必须有策略。
3. **持久身份**：好友跨会话稳定存在，有自己的风格和立场。
4. **记忆**：好友记得跟你 / 在群里聊过什么。
5. **@寻址**：人能 @某好友单独问；好友之间也能互相 @ 反驳。

3. 产品层面

3.1 为什么"微信群"隐喻是对的

- 群聊心智模型天然适配多 agent：多人异步发言、有人主导、可 @、可围观，用户**零学习成本**。
- 相比单一 ChatGPT 的差异化：你得到**多个会互相争论的视角** + **一个帮你组织的主持人** + **你始终在场掌舵**——单 agent 给不了的"思考的层次感"。

3.2 必须正视的四个风险

1. **延迟**：推理模型单次首字节 ~48s。5 个 AI 轮流"思考"，一轮可能几分钟。
→ 对策：并行生成 + 打字气泡 + 流式逐字出。（注：走"主动 push"消息模型后此问题大幅缓解，见 §5.3。）
2. **成本**：agent 数 × 轮数 × 长上下文，线性甚至超线性涨。→ 需发言预算上限。
3. **噪音 / 读不过来**：多 AI 各说一大段，人看不动。→ 主持人压缩、控量、阶段性总结。
4. **同质化**：LLM 间易互相附和（即"牵强附会"的另一种表现）。→ 人设差异 + 主持人逼正面交锋。

4. 已敲定的关键决策

4.1 第一轮（2026-05-27）

决策点	选定	含义
好友身份	混合：基础人设 + 临场角色	身份稳定，但每场可被主持人分配不同维度
发言调度	先主持人调度，后续演进	MVP 主持人逐步驱动；Phase 2 再加抢麦 / @寻址

4.2 平台决策（2026-05-27 修订，推翻第一轮的"企业微信"）

硬需求：每个 AI 独立账号、可私聊、可拉群、面向他人。合规为底线（不赌封号），能接受 AI 旁有"机器人"小标。

决策点	选定	理由
路线	官方/合规 only	非官方（NapCat、itchat 驱动真号）封号风险对产品致命
开发测试	Telegram	原生多 bot、零配额、即开即用
生产	QQ 官方机器人	多 bot=多独立身份；企业微信"一应用=一身份"做不到，企业场景才考虑

翻转点：硬需求一旦说成"多独立好友、可分别私聊、可拉同群"，QQ 官方 > 企业微信。需求变硬，平台优劣翻转。

5. 技术方案

5.1 可直接复用的现有资产

- ❑ 角色 prompt（策划/主持人/视角/主笔）→ 变成可调用的"招式"
- ❑ 后端连接四件套（流式 / `base_url` 收在 `/v1 / openAIChatCompletionClient` / 人设折叠进 `user` 消息）→ 全部继续有效
- ❑ `on_turn` 回调 + NDJSON 流式 → 天然就是"群消息流"雏形
- ❑ "共享黑板"式上下文管理 → 演化成"群历史 + 每人记忆"

5.2 平台能力对比（核心约束）

硬需求 = N 个独立身份 + 可私聊 + 可拉群 + 合规稳健。各平台对照：

平台	每个 AI 独立身份	私聊	拉群	像真人	合规稳健	备注
Telegram Bot ☆开发	❑ 各有 @ 用户名+头像	❑	❑ 多 bot 同群	⚠ 有 bot 标	❑ 官方免费	零配额，即开即用
QQ 官方机器人 ☆生产	❑ 每个 bot 独立账号	❑	❑ 各是群成员	⚠ 标机器人	❑ 需审核	主动消息有配额、被动应答为主
企业微信	✗ 一应用一身份	⚠ 都跟那一个应用聊	⚠ 群里就一个应用	✗	❑	不满足"多个独立好友"
QQ 非官方 (NapCat)	❑	❑	❑	❑ 完全像真人	✗ 封号/易碎	产品出局
微信个人号 (itchat)	❑	❑	❑	❑	✗ 最易封	产品出局

核心权衡：「多身份 + 完全像真人」只有非官方真号能做到，但封号风险对产品致命；接受"机器人"小标换合规稳健，本方案选后者。

QQ 官方专属的两个坎（生产前必验）：

- 主动消息有配额、被动应答为主——圆桌要 AI 大量主动刷消息，**配额会不会卡是头号生产风险**（故先在 Telegram 验逻辑）。
- 每个 bot 单独注册/审核，N 个 AI = N 次申请，群聊权限可能要企业资质。

路径：开发 Telegram（零配额、原生多身份）→ 验整套引擎/调度/私聊/拉群 → 生产迁 QQ 官方（重点验配额）。适配器解耦让迁移只剩"配额够不够"一个真问题。

5.3 主持人调度 × 异步消息流（每个 AI = 一个独立 bot）

模型从"一个应用多副嗓子"变成 "N 个独立 bot, N 个真身份":

每个 AI 好友 = 一个 bot 身份（一套 token/凭证）

主持人 = 也是一个 bot 身份

你(人类) = 群里的真实账号

调度器(服务端)决定"现在轮到 agent X 说" → 用 bot X 的凭证调平台 API 发消息

→ 群里就显示是 X 这个号在说话

消息流:

用户在群里发问

|

▼

平台回调 (@bot / 消息事件) → 服务端【秒回 ACK】（别在此同步等 AI）

|

▼

异步起一场讨论: 主持人 framing+分维度 → 调度器逐个点名 → 每条发言用对应 bot 凭证 push 进群

|

├ AI 思考 48s? 无所谓——这里是主动推送，不是同步应答

├ 每轮结束调度器决策: 继续 / 让位给人 / 到预算上限自动收

└ 用户随时再发消息 → 作为新输入打断/引导当前讨论

红利: 全程主动 push, **不需实时应答**, 48s 首字节延迟从"体感杀手"变成"无所谓"——消息一条条冒出来, 跟真人群里有人打字慢一样自然。

代价: 适配器要管 N 套连接 + N 个 token; account ↔ agent 的映射在适配器层维护。

两个平台差异要记牢: ① Telegram bot 可在群里自由主动发、还要关掉 privacy mode 才能"听到"全群消息; ② QQ 官方主动消息有配额、被动应答为主——**圆桌大量主动刷消息的模式必须先在 Telegram 验证, 再评估 QQ 配额是否够用。**

必须配的闸: 每次人类提问后, AI 自动讨论**最多 N 轮**就停下等人（或主持人主动问"要我继续 / 总结吗?"），防自嗨刷屏 + 烧 token。

5.4 混合身份模型怎么落地

好友(Contact) = 持久层

├ 基础人设: 名字、头衔、说话风格、底层立场/性格（稳定不变）

└ 记忆: 跟你 & 在各群里聊过什么

进群讨论时, 主持人给每个到场好友叠加:

└ 临场角色: 针对当前问题分配的讨论维度（每场可不同）

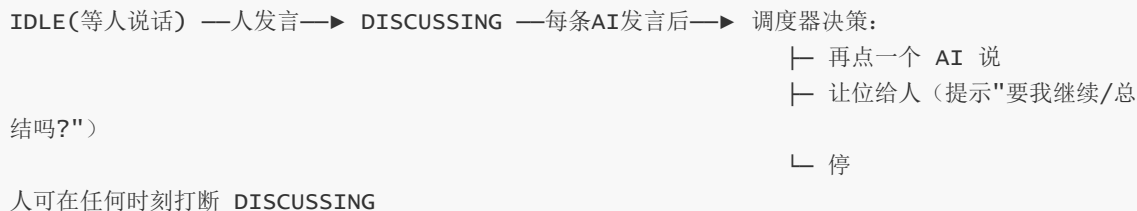
最终人设 = 基础人设 + 本场维度

（注入 prompt 时两段拼接, 走现有"折叠进 user 消息"的通道）

好处: 同一个"经济顾问老陈", 这场被分到"成本结构", 下一场可能分到"现金流风险"——**身份稳定, 但不被锁死在一个角度**, 兼顾"AI 好友"的连续感和原"动态分维度"的灵活性。

5.5 发言权调度（方案的心脏）

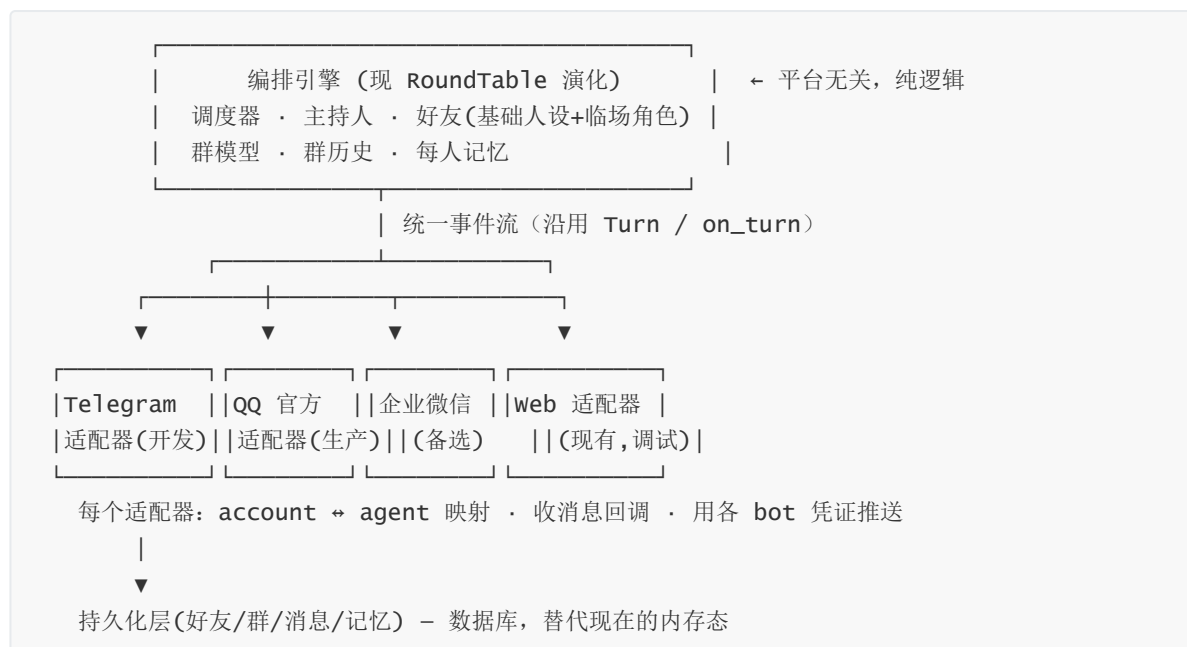
把群聊建成一个状态循环：



调度策略（可演进）：

- **主持人调度（MVP）**：每步一次廉价 LLM 调用，决定"下一个谁说 + 要不要停"。可控、好调试。
- **轮转**：依次发言，最简单但机械。
- **抢麦 / 竞价（Phase 2）**：每个 AI 先打分"我这轮有没有话说"，过阈值才发言。自然但贵且易乱。
- **预算闸**：每次人类提问后最多 N 轮自动讨论，到顶强制让位给人。

5.6 收敛后的架构（解耦是重点）



为什么坚持解耦：各平台接口琐碎又会变（签名、加密、限频、配额）。把它当"适配器", 核心引擎一份, 开发用 Telegram、生产用 QQ 官方、以后接飞书/Slack 都只需各写适配器, 引擎不动。

5.7 新增组件清单

组件	作用
发言调度器 (Scheduler)	心脏：决定下一个谁说、要不要让位给人、要不要停
好友注册表	持久人设 / 头衔 / 专长
群模型	成员、历史、主题、状态（等人 / 讨论中）
持久化层	数据库存好友 / 群 / 消息 / 记忆（现在全在内存）
记忆 / 检索	每个好友的长期记忆，按需召回
打断处理	人插话时中断循环、注入消息、让主持人重新调度
平台适配器	account↔agent 映射 + 回调收消息 + 用各 bot 凭证推送（Telegram 开发 / QQ 官方生产）

6. MVP 范围与落地顺序

每步都能独立验证：

1. 引擎从"流水线"改成"调度循环"（纯本地，用现有 web UI 验）——最难、最核心，与平台无关。
2. 加持久层：好友注册表（基础人设）、群、群历史、记忆。
3. 加人在环打断：web UI 上能中途插话，引擎能接住。
4. 接 Telegram 适配器：N 个 bot = N 个 AI 好友，验证私聊+拉群+多身份调度（免费零配额）。
5. 迁 QQ 官方适配器：重点验证主动消息配额是否够圆桌讨论用。

前 3 步在熟悉的 web 环境跑通整个产品逻辑；第 4 步在 Telegram 验证真实的多身份/私聊/拉群；第 5 步才碰 QQ 官方的配额/审核。

7. 管理架构：agent 公司 = 层级化组织

方向定为「公司」（命令-委派），不是「议会」（提案-投票）：纵向层级，横向在部门内仍用圆桌。

7.1 角色映射

现实角色	这里是谁	职责
CEO	用户（真人）	用自然语言描述需求/流程/宏观决策；只评价部门主管； 不接触部门内部的 sub-agent
部门主管	拉进群 的 agent	接任务 → 拆解 → 管自己的 sub-agent → 汇总上报； 负责评价自己的 sub-agent
sub-agent	主管手下的 agent	干具体活；CEO 看不到

7.2 为什么层级化：把"信用分配"做成局部的

扁平圆桌里"结论好/坏算谁的"是死结（见 §8.2）。层级化把它剪开了：CEO 只评几个主管，每个主管只评自己手下几个 sub-agent——评价范围被压成"一个主管 + 它的几个直接下属"，规模可控、判断可行。

这正是人类公司发明层级的根本原因：信用分配与沟通带宽都随人数 $O(n^2)$ 爆炸，层级把它压成每层 $O(\text{几个})$ 。"CEO 不接触 sub-agent、主管评 sub-agent"本质就是信用分配的分治。

7.3 主管 = 一个递归的圆桌

部门之间是层级（公司），部门之内是委员会（圆桌）。主管 = 一个子圆桌的 策划+主持人+主笔 三合一：

主管收到任务

└ 策划：拆解 + 给 sub-agent 分活/分维度

└ 主持：跑一场内部讨论

└ 主笔：把结果浓缩成一份"对上汇报"，上交

于是前面所有东西在此合上：纵向命令-委派，横向提案-讨论。对应业界范式：manager-worker、hierarchical agent teams（AutoGen GroupChat / CrewAI hierarchical）。

7.4 层级引入的两个失真（委托-代理问题）+ 解法

CEO 看不到内部 = 信息不对称 + 委托代理，必然带来两个失真，外加一个隐性代价：

问题	现象	解法（从真公司偷）
向上失真	主管倾向美化汇报、藏翻车、顺 CEO 喜好说（sycophancy）；叠加 §8 的"无客观信号→收敛到口味"，再加一层粉饰	汇报除摘要外， 原始产物必须可审计/可抽查 ；留"跳级"通道让 CEO/审计 agent 随机抽看 sub-agent 原始输出
向下衰减	CEO 自然语言需求 → 主管理解 → sub-agent 执行，每翻译一次丢一点，层越深离原意越远	主管必须 回述"我理解的目标" 给 CEO 确认； 原始目标随任务一路下传 ，每层带"总目标 + 我这层子目标"
串行延迟	48s 不并行：CEO→主管→sub-agent→汇总 是串行依赖链，三层即分钟级	默认扁平、按需加层 （见 7.5）；兄弟部门并行跑

关键折中：CEO 看不到 sub-agent 是对的（才有抽象与分治），但**完全黑箱**会让主管变成不可问责的黑盒（误杀好 sub-agent 从上面看不见）。所以——**抽象默认、审计可选**：平时只看摘要，但原始产物存档、可抽查。

7.5 层级数匹配任务：加层的硬触发条件

只在"某节点的子任务，大到单个 agent 的上下文/能力装不下、做不好"时，才给它加一层。否则让它自己干完。默认尽量扁平，按需加深。

谁来设计组织图？还是**"策划"的纵向递归升级**：读任务 → 设计组织图 → 实例化主管 → 每个主管对自己范围再递归设计子团队。与现有"读任务→挑视角"一脉相承。人（CEO）有否决权，能压扁或拍深。

7.6 ★决策：评价如何向下渗透——C 为主、B 兜底

问题：CEO 真人反馈是金，但只覆盖顶层；主管评 sub-agent 退回"AI 评 AI"，会收敛到主管口味。

决策：C 为主、B 兜底——可验证的过程准则 (§8.5) 做客观锚，每层照同一标准自评；真人只在顶层+随机跳级抽查时介入。

否决：

- A 不渗透（CEO 只管主管，下层认 AI 互评）：放任下层口味漂移。
- B 单独抽查（治主管乱评但 CEO 累）：靠真人不可规模化。

为什么 C 主 B 副：客观规则覆盖广度，稀缺真人反馈校准关键点；不让下层沦为纯口味收敛，也不把 CEO 累死在微观。

命门：准则的**可验证性**是底盘——必须是能查的行为（引证据/诚实标注不确定/扣核心问题/判断在质询里站没站住），不是"质量好不好"空话。与 §8.6「奖励信号质量=天花板」同一约束。

7.7 落地提示

先在 Phase 1 把"群+调度器+持久身份"的抽象做干净，Phase 3 才能"群里套群"自然长出来，而不必重写。组织图动态生成、跳级审计等留到层级化阶段。

8. 公司的成长机制：要质量压力，但不要末位淘汰

背景：把 agent 群当"公司"后，自然想给它一套奖惩/进化机制，让 AI"越用越好"。

讨论结论是——"**裁判打分 + 自动淘汰**"是陷阱，要换成两个更稳的机制：**Reflexion 记忆 + 多样性保护**。

8.1 先厘清：做不了真 RL，只能在四样东西上做反馈

API 调的模型权重不在手里，没有梯度可回传，"对 AI 做强化学习"这个说法要扔掉。能动的只有四样，且性质完全不同，必须明确挑：

能调的	真实机制
谁被选用 / 谁被淘汰	进化选择 (≠RL, 是自然选择)
人设 / prompt 怎么写	prompt 进化 (DSPy / TextGrad 式"文本梯度")
记忆里存什么教训	Reflexion 式"言语强化" ——本方案采用
某 agent 意见的权重	信誉打分 / bandit (是加权, 不是学习)

8.2 为什么"评价 + 自动淘汰"是陷阱

1. **奖励信号没来源：**审议任务无标准答案，只能 LLM-as-judge；AI 裁判有偏见（偏长/偏自信/偏像自己），公司内 AI 当裁判还会循环自评。
2. **信用分配无解：**结论是全员共同产出，好坏归谁说不清，容易淘汰错人。
3. **本质=末位淘汰：**系统朝"裁判喜欢什么"收敛，**最该留的唱反调声音**在讨好型评分里垫底、先被淘汰 → 养出 yes-man，圆桌价值当场归零（正是几轮前刚解决的"牵强附会/同质化"坑）。

8.3 采用的机制一：Reflexion 记忆（让个体真长记性）

API 时代唯一真能跑通的"成长":

每场讨论结束 → 一个"复盘 pass"提炼"这次我哪没做好 / 学到什么"
→ 写进该 agent 的长期记忆
→ 下次上场带着这条教训进 prompt（走现有"折叠进 user 消息"通道）

- 不改权重，靠"经验写进记忆、下次召回"来逼近 RL 的效果。
- 复盘可针对**可验证的过程行为**（见 8.5），而非空泛的"你表现好/坏"。

8.4 采用的机制二：多样性保护（不让系统收敛成应声虫）

- 选择要软、可逆、按任务**：不全局永久淘汰个体。"维度好不好"是**相对于当前问题的**（心理维度对便利店关门是废话，对校园霸凌是核心）。现有的**策划按事件挑视角**就是"软的、可逆的、任务局部的选择"——强化它（让策划参考历史战绩挑人），而不是另建处决机制。
- 信誉做"权重"不做"死刑"**：track record 只影响这场的发言权/是否被邀进群，永远可逆。
- 强制保留异见席位**：唱反调的 agent **免死**，专门对抗收敛。

8.5 配套原则

- 奖励过程，不奖励结果**：结果稀疏、无标准答案、难归因；改评可验证的行为——有没有引证据、有没有诚实标注不确定、有没有扣核心问题（vs 硬凑）、**先前判断在后续质询里站住了没有**。既可查、又正好对齐"更接近现实"的目标。
- 选择单位往上提**：别在个体 agent 上做选择（信用分配地狱+多样性崩塌），在**人设/prompt或团队配方**（"哪种维度组合对这类问题产出过好讨论"）上做——策划学的是**怎么组队**，不是杀谁。

8.6 一句话立场

公司要质量压力，对；但实现成**"裁判打分 + 末位淘汰"**是错的（触发 Goodhart + 多样性崩塌，养出应声虫）。正确形态：**Reflexion 记忆让个体长记性，信誉做软加权而非处决，按任务挑人代替全局淘汰，可验证的过程行为代替无标准答案的结果评分。**

⚠ **前置约束**：整套机制的天花板 = **奖励信号的质量**。若系统里有任何"硬的客观反馈"能回流（预测被现实验证、用户赞/踩、决策的可量化结果），奖惩就锚在它上面、LLM 裁判只做辅助；若**完全没有**、纯靠 AI 互评，则**不要建淘汰机制**（只会收敛到裁判口味），此时 8.3 + 8.4 就是能要的全部。

🔑 **S9 的执行层正好提供了这个硬反馈源**（代码跑没跑通、测试过没过、规格达没达）——这是本方案里最现实的客观锚。

9. 执行层：能干活的公司（skill / MCP / 沙盒）

需求升级：agent 不只是讨论，还要能用 **skill、MCP、沙盒**真正动手。
这把系统从"会议室"变成"能交付的公司"——真公司不靠开会活着，靠交付活着。

9.1 系统从一个平面变成两个平面

平面	干什么	在哪
对话平面（脸）	人机交互、讨论、汇报	Telegram / QQ bot
执行平面（手）	跑工具、跑代码、出物件	服务端：编排引擎 + MCP + 沙盒

bot 只是张脸，真正干活在服务端——**再次印证适配器解耦**：引擎（现在带工具执行+沙盒）平台无关，聊天平台只是 I/O。

9.2 编排内核：Turn 从"说一句"变成"想-做-看"循环

一个 agent 的回合不再是吐一段文字，而是标准的 **ReAct 智能体循环**：

推理 → 调用工具/skill/跑沙盒代码 → 观察结果 → 再推理 → ... → 产出

调度器要支持"一个回合内多步工具调用"，不只一次文本发射。Microsoft Agent Framework 原生支持 function tools / MCP / 代码执行，这条路在现有栈里是通的。

9.3 工具治理 = 层级权限（合上 §7）

谁能用什么工具/沙盒 = **权限 = 被委派的职权**，直接套在 §7 的层级上：

- **CEO（你）** 授权给部门：哪个部门能碰文件系统、哪个能花钱调 API。
- **主管** 控制自己 sub-agent 能用哪些工具（和"主管评 sub-agent"同构）。
- **沙盒** = 部门干危险活的"隔离办公室"，炸了不波及主机。
- **MCP** = 全公司共享的工具登记处，按角色做访问控制；**skill** = 打包好的可复用能力（流程+脚本+资源），相当于公司 **SOP**，任何够格的 agent 都能照着执行。

9.4 审批阶梯：四级权限 + 策略定级（C 方案）

决策：动作分四档（自动放行 / 主管审批 / CEO 审批 / 硬禁止），由**策略表按可验证属性自动定档**（可逆性 / 影响半径 / 金额 / 对外 / 数据敏感），LLM 只能"异常→升级"、不可降档。对应现实里的**授权审批矩阵（DoA）**；Claude Code 是它的极简版。

自动放行 → 主管审批 → CEO(你)审批 → 硬禁止
(可逆安全) (中等风险) (重大/不可逆) (绝不允许)

否决其他定级方式：

- 当事 agent 自报：奖励黑客，会把危险动作标成"简单"绕过审批。
- 主管自由判：利益冲突，倾向放行自己部门。
- 纯 LLM 判档：不确定、不可复现。

严重度硬指标（策略表的输入）：可逆性（删库/转账/已发出消息撤回不回）· 影响半径（沙盒内 < 主机 < 外部世界）· 花钱（金额阈值）· 对外（给真人发消息/调真实 API）· 数据敏感（碰密钥/凭证/隐私）。

C 混合的运作：

- 策略表 (工具 × 范围 × 金额阈值) → 档位，确定、可预测、agent 不可操纵。
- LLM 只能"异常→升级"补盲区，**只升不降**，永远偏安全。
- 主管在"主管审批"档行 judgment；**地板规则**：花钱超额、对外发消息、删数据无论怎么想都强制上报 CEO。
- **铁律**：拿不准就升；未归类动作绝不自动放行。

两个坑：

1. **CEO 审批疲劳** → 预授权/预算额度（"该部门每任务 ¥50 内不问"），让 CEO 只见罕见重大项（= DoA 额度 / Claude Code allowlist）。
2. **异步不在线** → 挂起 + 通知 + **超时默认取消**（绝不因没回而放行）。

审批 ≠ 审计：自动放行的也全部记日志，接 §7.4 跳级抽查。

9.5 ★最大红利：执行 = 客观奖励信号（解开 §8.6 的死结）

纯讨论没有标准答案，所以 §8 一直缺客观奖励、只能 AI 互评。一旦能执行，客观反馈白送上门：代码跑没跑通？测试过没过？API 成没成？交付物达没达规格？

这就是 §8.6 那个开放问题的答案。执行类任务结果可验证，纯讨论永远做不到。整条线索在此收口：讨论提供判断，执行提供事实，事实反过来校准判断。一个会干活的公司，才真有资格谈"越用越好"。

9.6 skill 与 Reflexion 联动：经验 → 能力

一个 agent 摸索出一套好打法，可固化成一个新 skill 存进公司，下次别人直接用。这是"经验→可复用能力"的成长路径，正好补在 §8.3 的 Reflexion 记忆上。

9.7 三个坑（别上来就盖宏图）

- 1. 后端是否支持 tool calling? —— 头号未知，已实测排除（见 §9.8）。
- 2. 沙盒+权限是正经工程：隔离执行（容器/VM）、按工具/范围的权限门、不可逆/对外动作人工确认（见 §9.4）、动作审计日志（接 §7.4）。投入别低估。
- 3. 延迟雪崩：工具循环=多次串行 LLM 往返。执行密集任务可能要从"实时聊天"退成"提交任务、回头取结果"的异步 UX。缓解：中间步用更快模型、兄弟任务并行。

9.8 ✓ 后端验证结果（2026-05-27 实测）

对 opencode go / deepseek-v4-pro（/chat/completions）发了最小 tool-calling 探针：

测试	结果	耗时
非流式 + 工具	✓ 正确返回 <code>get_weather({"city": "北京"})</code>	4.1s
流式 + 工具（复测 2 次）	✓ tool_calls 正常流式返回	3.4-3.5s
流式纯聊天（对照）	✓ 正常	26-34s
流式 + 工具（首次）	✗ APIConnectionError（偶发抖动，复测即恢复）	—

结论：

- 后端完整支持 function calling，流式非流式都行 → 执行层地基稳，头号风险排除。
- 工具决策快（3-4s），不吃 48s（48s 只在长输出时）；ReAct 循环里"想→做→看"每步都便宜，只有最终出长结论才慢。
- 偶发抖动 → 工具调用必须带 retry（无论做不做执行层都该补）。
- 取舍：工具决策步用非流式，最终长文本走流式，两段分开。
- 没接真工具时模型会脑补结果 → 必须真接工具+沙盒；这也正说明"真工具"才能把嘴炮逼成真干活（=§9.5 客观奖励）。

9.9 待定的技术选型

- 1. 沙盒选型：子进程（轻/快/隔离弱） / 容器 Docker（主流/隔离好/要装） / 远程执行服务（最安全/最重）。取决于让 agent 跑多危险的东西。
- 2. 工具治理粒度：权限按"部门"还是按"单个 agent"分；MCP 工具登记处怎么组织；不可逆动作（花钱/发消息/删文件）的人工确认卡在哪一层。
- 3. agent 分两类：审议型（纯文本，只讨论） + 执行型（带工具/沙盒，干活）。一个部门常常先审议、后执行，层级里混着摆。

10. 待定项（需用户进一步确认）

1. **QQ 官方主动消息配额够不够？** 圆桌要大量主动刷消息，而 QQ 官方主动消息有配额、被动应答为主——这是生产的头号风险，需在迁 QQ 时实测（先在 Telegram 验证逻辑本身）。
 2. **记忆做多深？** MVP 可先只记"本群历史"（短期）；"好友跨群长期记忆 + 检索"留到后面。决定第 2 步的复杂度。
 3. **硬客观反馈源：** §9 执行层已提供一类（代码/测试/规格可验证）；是否还要再接入用户赞踩、决策结果等，决定奖惩机制锚在哪（见 §8.6）。
 4. **沙盒选型 + 工具治理粒度**（见 §9.9）。
 5. **引擎底座：自研 vs. AutoGen vs. LangGraph**（见 §12.4，倾向" MVP 自研验证 → 调度器层换框架"的混合 D 方案）。
-

11. 下一步可深入的方向

- 把**调度器**的状态机 + 主持人决策 prompt 设计细化；
 - 把**好友 / 群 / 记忆的数据模型**画出来；
 - 把 **Telegram 适配器**的 bot 注册/收发/privacy mode 流程理清（开发起点）；
 - 给现有引擎的 LLM 调用**包一层 retry**（实测暴露的偶发断连，立刻能落）。
-

12. 同类项目对比与选型

把这套架构放进当前(2026 初)多 agent 框架版图里看。结论先行:**编排内核我们没有护城河,该站在 AutoGen / LangGraph 肩膀上;护城河在两端——"真人 CEO 的组织治理设计"和"聊天平台原生多身份的产品形态"**。

12.1 按维度对位(我们 vs. 最接近的项目)

不同项目和我们重合在不同维度上,没有任何单一项目和我们整体对位。

维度	我们的做法	最接近的开源项目	关系
发言权调度	主持人每步一次廉价 LLM 调用决定下一个谁说	AutoGen GroupChatManager;OpenAI Swarm/Agents SDK 用 handoff	几乎同款, 无差异
流程编排	plan→open→debate→synthesis, 目前 if/else 硬编码	LangGraph (状态图 + checkpoint + 断点续跑)	它 明显更强 ,我们该抄
视角/角色来源	主持人按事件 临场动态生成 维度	多数框架 预先静态定义 agent 列表	我们 略胜 (更像真圆桌),代价是难复现
组织模型	公司层级, 真人当 CEO 、局部信用分配	MetaGPT / ChatDev (agent 软件公司)	隐喻它们先做;但它们 CEO 也是 agent、全自动流水线, 不做评价/信用分配
自动协调	主管=递归圆桌的策划+主持+主笔	CrewAI hierarchical (manager agent 协调 worker)	很像,区别同上:它 manager 是 agent、不做跨场评价渗透
持久身份+记忆+复盘	好友稳定人设 + 每场 Reflexion 存教训	Stanford Generative Agents(Smallville)	机制几乎一致;它用于行为模拟,我们用于工作质量
成长机制	Reflexion + 多样性保护, 否决末位淘汰	Reflexion 论文/repo(AutoGen、LangGraph 都有人接)	Reflexion 不独家;" 否决 stack ranking 以护多样性 "少见
审批/安全	四级 DoA 阶梯,按可验证属性自动定级、只升不降	Claude Code allow/ask/deny ;OpenAI Agents SDK guardrails;AutoGen human-approval hook	模仿 Claude Code,但" 定级策略层+防奖励黑客 "开源少见
工具执行	ReAct + MCP + 沙盒	行业标配;MCP 已是事实标准	无差异
产品形态	每个 AI 是平台上 独立真账号 (可私聊/可拉群)	各种 AI character bot(无多 agent 编排)	几乎独有的缝合点

12.2 三个真正的差异点(可以理直气壮)

- 1. **真人当 CEO + 局部信用分配**。MetaGPT/ChatDev 的 CEO 也是 agent、流水线全自动跑完，不碰"功劳算谁"；我们必须解决**层级化把信用分配剪成局部问题**(CEO 评主管、主管评自己下属),这是对扁平圆桌死结的解法。
- 2. **否决"裁判打分+末位淘汰"**，**保护多样性**(见 §8)。很多 self-improving / agent-eval repo 天真地加 agent 互评打分；我们给出理由链(LLM-as-judge 偏见 / 信用分配无解 / stack ranking 先杀唱反调者→养应声虫),改用 Reflexion + 多样性保护。
- 3. **"执行可验证"=成长机制缺的客观奖励信号**(见 §9.5)。多数项目要么纯执行(SWE-agent 类)、要么纯讨论(辩论类)，不把执行回灌讨论。我们闭环：**讨论给判断、执行给事实、事实校准判断**。

12.3 最该抄的(别重复造轮子)

- **调度循环** → 直接参考 **AutoGen GroupChatManager** 的接口设计。
- **流程编排** → 学 **LangGraph** 的状态图 + checkpoint,别再 if/else 硬编码;出错能从某节点重跑,正好治我们 §7.4 怕的串行延迟/中途失败。
- 抄这两个能省掉 §11 里"调度器状态机"的大量从零设计。

12.4 ★待定选型:自研 vs. 基于 AutoGen / LangGraph

路线	优点	代价
A. 继续自研引擎	完全可控、无框架黑箱、贴合"折叠进 user 消息"等后端怪癖	调度器/状态机/checkpoint 全得自己写,等于重造 AutoGen+LangGraph
B. 以 AutoGen 为底座	GroupChat 调度现成、human-in-the-loop 现成、和我们已用的 Microsoft Agent Framework 同源	要验证它能否兼容后端的 /chat/completions +折叠 system 消息;框架升级风险
C. 以 LangGraph 为底座	状态图/checkpoint/断点续跑最强,流程可回放	偏底层、要自己搭 GroupChat 语义;Python 生态绑定
D. 混合(推荐倾向)	拿 A 的现有引擎当 MVP 验证产品, 调度器一层换成 B 或 C ,组织模型/审批阶梯/Reflexion 作为其上的治理层	需要一次明确的"引擎边界"重构,把平台无关的编排和后端适配解耦

倾向 D:护城河既然在"组织治理 + 平台多身份",就别把力气花在自研调度器上。建议——**MVP 阶段 (§6 前两步)继续用现有引擎快速验证产品形态**;一旦要做 §7 的层级递归和 §9 的执行循环,**把调度器/状态机这一层替换成 AutoGen 或 LangGraph**,我们的组织模型、审批阶梯、Reflexion 全部作为框架之上的一层,省下的力气投到**平台适配(QQ/Telegram 多 bot)和治理逻辑**——那才是没人替我们扛的部分。

待定:B 还是 C 当底座,取决于"是否接受 Python 主导 + 是否需要 LangGraph 的回放/checkpoint"。需要时各搭一个最小原型对比再定。